

Dubai DB2 project pipeline column documentation

Source files:

Projects:

D:\Dubai\Dubai_Updated_data\Raw_Data\Projects.csv

Units:

D:\Dubai\Dubai_Updated_data\Project_Unit_Summary\Units_with_actual_area_sqft.csv

Buildings:

D:\Dubai\Dubai_Updated_data\Raw_Data\Buildings.csv

Developers:

D:\Dubai\Dubai_Updated_data\Raw_Data\Developers.csv

This pipeline creates the DB2 project output by:

1. Loading project-level records from Projects.csv.
2. Aggregating unit-level data by project_id from Units_with_actual_area_sqft.csv.
3. Creating date, unit-count, area, BHK, subtype, building, and developer derived columns.
4. Renaming/mapping those columns into the final DB2 schema.
5. Creating DB2 schema columns that are not available from the raw sources as blank/None columns in the final output.

Project id normalization

_pid_norm:

Source: project_id.

Logic: normalize_project_id removes numeric formatting differences so joins work consistently. For example 123.0 becomes '123'. Blank or missing values become None.

Purpose: used as the merge key between project records and unit aggregates.

Unit aggregation columns

These columns are created from the Units file, grouped by normalized project_id.

units_rooms_en:

Source: rooms_en.

Logic: count each room label per project. Missing or blank room labels are counted under 'NA'. Stored as a dictionary like {'1 B/R': 10, '2 B/R': 4}.

units_actual_area:

Source: actual_area_sqft.

Logic: count each actual area value per project. Numeric areas are formatted to two decimals before counting. Missing or blank areas are counted as 'NA'.

units_property_sub_type_en:

Source: property_sub_type_en.

Logic: count each property subtype per project. Missing or blank subtypes are counted as 'NA'.

units_project_name_en:

Source: project_name_en from the Units file.

Logic: keeps the first non-null English project name found for the project.

units_project_name_ar:

Source: project_name_ar from the Units file.

Logic: keeps the first non-null Arabic project name found for the project.

unit_count:

Source: Units rows.

Logic: total number of unit records found for the project. Missing aggregate values after merge are filled as 0 and converted to integer.

rooms_area_dict_raw:

Source: rooms_en and actual_area_sqft.

Logic: nested dictionary by room label and area value. It counts how many units exist for each room/area pair. Missing rooms become 'NA'; missing areas become 'NULL'.

Date and quarter columns

project_start_date, project_end_date:

Source: raw project_start_date and project_end_date.

Logic: converted to pandas datetime using format '%d-%m-%Y'. Invalid values become NaT because errors='coerce' is used.

Start_Year:

Source: project_start_date.

Logic: extracts year as nullable Int64.

End_Year:

Source: project_end_date.

Logic: extracts year as nullable Int64.

Start_Quarter:

Source: project_start_date.

Logic: builds 'Q<quarter> <year>', for example 'Q1 2020'. Blank string is used when the start date is missing or invalid.

End_Quarter:

Source: project_end_date.

Logic: builds 'Q<quarter> <year>'. Blank string is used when the end date is missing or invalid.

Start_Quarter_Year:

Source: project_start_date.

Logic: pandas quarterly period version of the start date.

End_Quarter_Year:

Source: project_end_date.

Logic: pandas quarterly period version of the end date.

Bedroom/unit category columns

categorized_units:

Source: units_rooms_en dictionary.

Logic: categorized_units maps room labels into standard buckets:

- '< 1 B/R' when room label contains 'SINGLE ROOM' or equals 'SINGLE'.
- 'PENTHOUSE' when room label contains 'PENTHOUSE'.
- 'Commercial' when room label contains GYM, HOTEL, KIOSK, OFFICE, SHOP, or STUDIO.

- '1 B/R' through '5 B/R' when a bedroom count is parsed from labels like 1 B/R, 2BR, or 3 BHK.
- '> 5 B/R' when parsed bedroom count is greater than 5.
- 'Other' when no rule matches.

< 1 B/R, 1 B/R, 2 B/R, 3 B/R, 4 B/R, 5 B/R, > 5 B/R, PENTHOUSE,

Commercial, Other:

Source: categorized_units.

Logic: each standard bucket is expanded into its own integer count column.

Missing buckets are filled as 0.

Area columns

units_actual_area_Cumulative:

Source: units_actual_area dictionary.

Logic: multiplies each area key by its unit count and sums the result.

Non-numeric area keys are skipped. Result is rounded to 2 decimals.

rooms_area_dict:

Source: rooms_area_dict_raw.

Logic: JSON string version of the raw nested room/area/count dictionary.

Empty dictionaries are written as '{}'.

rooms_area_dict_categorized:

Source: rooms_area_dict_raw.

Logic: room labels are normalized into internal buckets:

lt_1_br, 1_br, 2_br, 3_br, 4_br, 5_br, gt_5_br, PENTHOUSE,
Commercial, Other.

Area values are formatted to two decimals and counts are summed by bucket.

The final value is stored as JSON.

carpet_area_<bucket> columns:

Source: rooms_area_dict_categorized.

Logic: for each bedroom/commercial bucket, area * count is summed. Dynamic columns are created only for buckets present in the data, such as

carpet_area_1_br or carpet_area_Commercial.

Property subtype columns

Flat, Shop, Office, Other:

Source: units_property_sub_type_en.

Logic:

- Flat includes FLAT, UNIT, BUILDING.
- Shop includes SHOP, SHOW ROOMS, STORE.
- Office includes CLINIC, OFFICE, WAREHOUSE, WORKSHOP.
- Other includes every other subtype or unexpected value.

Counts are summed into the four columns.

property_type_flag:

Source: Flat, Shop, Office, Other counts.

Logic:

- 'Residential' when Flat > 0 and Shop/Office/Other are all 0.
- 'Residential + Commercial' when Flat > 0 and any other category > 0.
- 'Commercial' when Flat == 0 and any Shop/Office/Other category > 0.
- 'NA' when no category has a positive count.

Final DB2 column: project_type.

subtype_actual_area_dict:

Source: raw Units file grouped by project_id, property_sub_type_en, and actual_area_sqft.

Logic: sums actual_area_sqft by subtype. Missing or invalid areas are counted in a '___NULL_COUNT___' entry instead of being added to area totals.

Stored as JSON.

carpet_area_Flat, carpet_area_Shop, carpet_area_Office, carpet_area_Other:

Source: subtype_actual_area_dict.

Logic: sums subtype-level actual area into the same four property subtype groups. '___NULL_COUNT___', 'NULL', and 'NA' keys are ignored.

Building columns

floor_list:

Source: Buildings.csv floors grouped by project_id.

Logic: converts building floors to numeric, fills missing as 0, rounds to integer, and stores the list of floor counts per project.

building_data:

Source: Buildings.csv project_id, building_number, shops, flats, offices.

Logic: creates a JSON dictionary for each project with each building number as a key and shops/flats/offices counts as values.

_building_floors_dict:

Source: Buildings.csv building_number and floors.

Logic: helper dictionary mapping each building number to its floors count. Used to build tower completion and sanctioned floor JSON columns.

carpet_wise_total_sold_units:

Source: units_project_name_en and rooms_area_dict.

Logic: builds JSON in the form {project_name: [[room_area_dict]]}. Each area count is converted to [count, 0], where 0 represents unsold/booked placeholder logic used by the existing downstream format.

bhk_wise_total_sold:

Source: units_rooms_en.

Logic: for each room label, creates {'total': count, 'sold': 0, 'unsold': count}. Stored as JSON.

Final DB2 column: carpet_wise_total_booked_units.

commencement_date:

Source: project_start_date and units_project_name_en.

Logic: formats start date as dd/mm/yyyy and stores JSON {project_name: [formatted_start_date]}. Missing date/name returns None.

final_proposed_date_of_compeletion:

Source: project_end_date and units_project_name_en.

Logic: formats end date as dd/mm/yyyy and stores JSON {project_name: [formatted_end_date]}. Missing date/name returns None.

Final DB2 column: final_proposed_date_of_completion.

project_wise_bhk_info:

Source: rooms_area_dict and units_project_name_en.

Logic: for each room type, calculates total unit count and weighted average

area. Stored as a project-keyed dictionary string.

Final DB2 column: project_bhk_summary.

bhk_wise_ca:

Source: rooms_area_dict and units_project_name_en.

Logic: collects unique carpet area values per room type, lowercases room labels, and stores a project-keyed JSON string.

Final DB2 column: bhk_wise_carpet_area.

project_wise_commencement_quarter_and_total_units:

Source: units_project_name_en, Start_Quarter, and unit_count.

Logic: groups by project and Start_Quarter, sums unit_count, sorts quarters, and stores JSON {project_name: [[quarter, total_units], ...]}.

Note: quarter_sort_key expects 'Qn-yyyy', while Start_Quarter is built as 'Qn yyyy'. If the format does not match, the fallback sort key is used.

Final DB2 column: project_commencement_quarter_units.

bhk_wise_min_max:

Source: rooms_area_dict.

Logic: for each room type, extracts numeric area keys and stores [minimum_area, maximum_area] as JSON.

Final DB2 column: bhk_wise_min_max_area.

Building_count:

Source: building_data JSON.

Logic: parses building_data and returns the number of top-level buildings.

Invalid or blank JSON returns 0.

Final DB2 column: total_building_count.

project_tower_completion_date:

Source: units_project_name_en, completion_date, and _building_floors_dict.

Logic: completion_date is parsed and formatted as dd-mm-yyyy. If project name or building data is missing, an empty nested project JSON is returned.

Otherwise each building number is assigned the same completion date.

number_of_sanctioned_floors:

Source: units_project_name_en and _building_floors_dict.

Logic: creates JSON {project_name: {building_number: floor_count}}.

Missing project name returns '{}'; missing floors returns {project_name: {}}.

Developer columns

organization_individual:

Source: Developers.csv legal_status_en mapped by developer_id.

Logic: if Developers.csv exists and contains developer_id and legal_status_en, the first developer_id in the project developer_id value is used to look up legal_status_en. Otherwise value is None.

Final DB2 column: organization_individual_name.

number_of_developers:

Source: developer_id.

Logic: splits developer_id by comma and counts the entries. Blank or NaN values return 0.

Note: the final mapping currently points from 'no_of_developers' to 'number_of_developers', while the derived column is named 'number_of_developers'. Because of that name mismatch, the final output

column is created as None unless a source column named no_of_developers exists.

Final DB2 mapped columns

These output columns are created by the final mapping. When the source column is available, its values are copied/renamed. When the source is None or missing, the output column is created with None.

```

index                <- project_number
registered_project_name    <- units_project_name_en
project_name           <- units_project_name_en
project_name_ar        <- project_name before overwrite
location_name          <- area_name_en
city_name              <- city_name, later fixed as Dubai
project_latitude        <- Latitude
project_longitude      <- Longitude
location_latitude       <- area_latitude
location_longitude      <- area_longitude
plot_number            <- None
project_registration_id  <- project_id
is_coordinate_manually_done <- Geocode_Status
total_units             <- unit_count
booked_units            <- total_sold
commencement_date       <- commencement_date
building_wise_total_booked_units <- None
final_proposed_date_of_completion <- final_proposed_date_of_completion
project_bhk_summary     <- project_wise_bhk_info
project_commencement_quarter_units <-
  project_wise_commencement_quarter_and_total_units
organization_individual_name <- organization_individual
number_of_developers    <- no_of_developers
pincode                 <- None
registered_project_count <- None
remark                  <- None
total_fsi                <- None
total_plot_area_sq_m     <- None
bhk_wise_min_max_area    <- bhk_wise_min_max
bhk_wise_carpet_area     <- bhk_wise_ca
project_type             <- property_type_flag
bhk_wise_total_booked_units <- None
carpet_wise_total_booked_units <- bhk_wise_total_sold
total_building_count     <- Building_count
project_tower_completion_date <- project_tower_completion_date
number_of_sanctioned_floors <- number_of_sanctioned_floors
amenity_profile          <- amenity_profile
age_of_project           <- age_of_project
construction_status      <- construction_status
building_grade           <- building_grade
zoning_type              <- zoning_type
encumbrance_status       <- encumbrance_status
country_name             <- country_name, fixed value
state_name               <- state_name, fixed value
sub_locality             <- sub_locality

```

micro_market	<- micro_market
frontage	<- frontage
approval_status	<- approval_status
data_source	<- data_source
source_accessibility	<- source_accessibility, fixed value
source_accessibility_way	<- source_accessibility_way, fixed value
sourcing_cost	<- sourcing_cost
sourcing_time	<- sourcing_time
raera_location_v1	<- None
Old Rera Location	<- None
Super Modified Project Name	<- None
Rera Location - v2	<- None

Fixed values applied before final output

These columns are overwritten with constant values:

city_name	= 'Dubai'
state_name	= 'Dubai'
country_name	= 'United Arab Emirates'
source_accessibility	= 'Easy'
source_accessibility_way	= 'Mining'

Explicit project-name preparation before mapping

registered_project_name:

Source: units_project_name_en.
Logic: copied before final mapping.

project_name_ar:

Source: project_name at that point in the dataframe.
Logic: copied before project_name is overwritten.

project_name:

Source: units_project_name_en.
Logic: overwritten to the English project name from Units aggregation.